PRÁCTICA PROFESIONALIZANTE IV · ETAPA 3 · DESARROLLO COMPLETO

Plataforma web del IFTS N.° 2

Manual Técnico

EQUIPO

Talento Tech (IFTS N.° 29)

Santiago Cuda
Adrián Madroñal
Martín Juan
Nicolás Rolón
Gastón Zampar

MATERIA

Práctica Profesionalizante IV
Tecnicatura Superior en Desarrollo de Software

DOCENTES

Prof. Lic. Emir García Ontiveros
Prof. Lic. Kevin Del Bello

8 de noviembre de 2026 · Cliente: IFTS N.° 2 DE 20, Gobierno de la Ciudad de Buenos Aires

Índice

Manual Técnico **3**

1. Introducción general 3
2. Instalación y puesta en marcha 4
3. Arquitectura del sistema 7
4. Estructura del proyecto 11
5. Base de datos 13
6. Configuración inicial 30
7. Integraciones, despliegue y operación 31
8. Componentes principales 32
9. Gestión de errores y validaciones 34
10. Pruebas implementadas (resumen) 36
11. Limitaciones conocidas 36
12. Para continuar el proyecto 37

Manual Técnico

Plataforma web del IFTS N.° 2 — sitio institucional y panel de gestión. Equipo Talento Tech · Práctica Profesionalizante IV · Etapa 3.

Este manual está dirigido a quien tenga que **instalar, mantener o continuar** el desarrollo sin depender de los autores originales. Si solo querés ver el sistema andando, andá directo a la sección 2 (*Instalación y puesta en marcha*): son seis comandos.

1. Introducción general

1.1 El sistema

La plataforma moderniza el sitio institucional del **IFTS N.° 2 DE 20** (Tecnicatura Superior en Emprendimientos Gastronómicos, Gobierno de la Ciudad de Buenos Aires), que estaba en Google Sites, abandonado y sin panel de gestión. Tiene dos caras:

- **Sitio público** (aspirantes, estudiantes, familias): Inicio, Institucional (con formulario de contacto), La carrera, Estudiantes (ciclo vigente, horarios, mesas de examen y avisos), Ingreso (aspirantes), Inscripciones (alumnos) y Novedades.
- **Panel de administración** (personal no técnico del instituto): moderar publicaciones de Instagram, cargar publicaciones propias, avisos académicos, horarios y llamados a examen, archivo documental, contenido del sitio, usuarios y roles, e integraciones (Instagram y correo).

1.2 Objetivo

Que el personal del instituto mantenga el sitio **sin saber programar**, con contenido siempre al día y sin riesgo de perder información histórica.

1.3 Alcance de esta entrega

Aplicación completa y funcional (sitio público + panel + API + base de datos) con CRUD completo, pruebas automatizadas y documentación. **Queda fuera o sin verificar** (ver la sección 11, *Limitaciones conocidas*): conexión con Instagram real, envío de correo real y almacenamiento S3 real.

1.4 Reglas de negocio que gobiernan el código

1. **Nada se borra solo.** Lo vencido se oculta y se archiva. Eliminar es siempre una acción manual, con confirmación y auditoría.
2. **La vigencia se calcula al consultar** (estado + fechas), no con tareas programadas.
3. **Nada de redes llega al sitio sin aprobación humana.**
4. **El sitio sigue funcionando** si fallan Instagram, el correo o el almacenamiento.
5. **El producto es genérico**: el nombre del instituto, contactos y redes son configuración (semilla y panel), no están escritos en el código.

1.5 Dónde está cada apartado que pide la cátedra

Apartado de la consigna	Sección de este manual
Introducción general	1
Configuración e instalación	2 (guía paso a paso) y 7 (variables, integraciones, despliegue)

Apartado de la consigna	Sección de este manual
Arquitectura del sistema	3
Estructura del proyecto	4
Base de datos (DER, tablas, relaciones)	5
Componentes principales	8
Gestión de errores y validaciones	9
Pruebas implementadas	10 (resumen) y el documento *Evidencias de pruebas* (detalle con evidencias)

2. Instalación y puesta en marcha

2.1 Qué necesitás

Herramienta	Versión usada	Mínima	Para qué
Node.js	24	20	Ejecuta la API, la web y los scripts
pnpm	11.8	11	Gestor de paquetes del monorepo (`corepack enable` o `npm i -g pnpm@11`)
Docker Desktop (Engine)	cualquiera	—	Levanta PostgreSQL 17 en un contenedor (no hace falta instalar PostgreSQL)
Git	cualquiera	—	Solo si clonás el repositorio (con el `.zip` no hace falta)
Internet	—	—	`pnpm install` y la compilación de la web (descarga la fuente de Google Fonts)

Funciona en Windows (Git Bash o PowerShell), macOS y Linux. Para las pruebas de punta a punta se usa Cypress, que trae su propio navegador (Electron) y lo descarga la primera vez; para generar el PDF (`pnpm docs:pdf`) hace falta Google Chrome.

2.2 Obtener el proyecto

- **Desde el `.zip` de la entrega:** descomprimilo en una carpeta (por ejemplo `ifts2-plataforma`) y abrí una terminal dentro de esa carpeta.
- **Desde el repositorio Git:** `git clone <url-del-repositorio> ifts2-plataforma` y `cd ifts2-plataforma` . La URL figura en la sección *Enlaces* de la documentación.

El proyecto se abre con cualquier editor (se recomienda Visual Studio Code); no hay un "proyecto de IDE" que importar: es un monorepo de Node con `pnpm-workspace.yaml` .

2.3 Pasos (copiar y pegar)

```
cp .env.example .env     # PowerShell: Copy-Item .env.example .env
pnpm install             # instala las dependencias y genera el cliente de Prisma
pnpm db:up               # levanta PostgreSQL 17 en Docker (puerto 5433)
pnpm db:migrate          # crea las tablas (migraciones de Prisma)
pnpm db:seed             # carga los datos de demostración
pnpm dev                 # API en http://localhost:4000 y web en http://localhost:3000
```

Qué hace cada paso:

1. `.env` : el archivo `.env.example` ya trae valores de desarrollo listos para usar. El `.env` nunca se versiona (contiene secretos en producción).
2. `pnpm install` : instala todo el monorepo (`apps/api` , `apps/web` , `packages/shared`). pnpm 11 pide aprobar scripts de instalación; ya están aprobados en `pnpm-workspace.yaml` .
3. `pnpm db:up` : `docker compose up -d --wait ifts2-db` . Crea el contenedor `ifts2-db` (PostgreSQL 17, usuario/clave `ifts2` / `ifts2`) con dos bases: `ifts2` (desarrollo) e `ifts2_test` (pruebas). **Usa el puerto 5433**, no el 5432.
4. `pnpm db:migrate` : aplica las migraciones de `prisma/migrations/` (equivale a ejecutar el script SQL de creación de la base) y regenera el cliente.
5. `pnpm db:seed` : carga el contenido (idempotente: se puede repetir sin duplicar).
6. `pnpm dev` : levanta la API y la web en modo desarrollo con recarga automática.

> **Si el puerto 3000 o el 4000 están ocupados** (error `EADDRINUSE`): poné en `.env` otros puertos coherentes, por ejemplo `API_PORT=4100` , `API_ORIGIN=http://localhost:4100` y `APP_ORIGIN=http://localhost:3100` , y levantá la web con `PORT=3100` (por ejemplo `PORT=3100 pnpm dev` ; en PowerShell: `$env:PORT=3100; pnpm dev`). `APP_ORIGIN` debe coincidir con la dirección real de la web o el panel te mandará siempre al login.

2.4 Comprobar que funciona

Qué	Dirección	Qué deberías ver
Salud de la API	`http://localhost:4000/health`	`{"status":"ok","api":"ok","db":"ok",…,"demoMode":true}`
Sitio público	`http://localhost:3000`	La página de Inicio del instituto con datos de ejemplo
Panel de gestión	`http://localhost:3000/panel`	Te pide ingresar: usá una de las credenciales de la tabla siguiente

2.5 Credenciales de prueba (solo desarrollo y demostración)

Rol	Correo	Clave	Puede
SUPERADMIN (Superadministrador)	`admin@ifts2.local`	`Cambiame-123!` (`SEED_ADMIN_PASSWORD`)	Todo, incluido crear roles y definir sus permisos
ADMIN (Rectoría / Dirección)	`direccion@ifts2.local`	`Demo-2026-ifts2`	Todo salvo roles; crea usuarios (también otros administradores)
BEDELIA (Bedelía Académica)	`bedelia@ifts2.local`	`Demo-2026-ifts2`	Avisos, horarios, llamados, archivo, mensajes
EDITOR (Comunicación)	`editor@ifts2.local`	`Demo-2026-ifts2`	Publicaciones y moderación de Instagram

La semilla **demo** crea el ciclo 2026 · 2.º cuatrimestre como público, avisos (vigente, urgente y vencido), publicaciones de Instagram detectadas, documentos archivados, el llamado «Turno extraordinario de Diciembre 2026», contenido de las páginas y enlaces. La semilla **prod** (`SEED_MODE=prod`) crea **solo** el administrador (con cambio de clave obligatorio al primer ingreso), los ajustes y los enlaces reales; nunca datos de demostración, y la semilla demo se niega a correr con `NODE_ENV=production` .

2.6 Crear, reiniciar y restaurar la base de datos

Necesidad	Comando
Crear la base y las tablas desde cero	`pnpm db:up && pnpm db:migrate` (los scripts SQL están en `prisma/migrations/*/migration.sql`)
Cargar datos de demostración	`pnpm db:seed`
Borrar todo y volver a empezar	`pnpm db:reset` (**destructivo**: pide confirmación a Prisma; ejecutalo una persona)
Hacer una copia de seguridad	`docker exec ifts2-db pg_dump -U ifts2 -d ifts2 -Fc > copia.dump`
Restaurar una copia en una base nueva	`docker exec ifts2-db createdb -U ifts2 ifts2_restaurada` y luego `docker exec -i ifts2-db pg_restore -U ifts2 -d ifts2_restaurada --no-owner < copia.dump`
Ver las tablas a mano	`docker exec -it ifts2-db psql -U ifts2 -d ifts2` (`\dt`)
Estudiar el modelo	`pnpm exec prisma studio` (explorador visual de la base)

Los archivos subidos (PDF, imágenes) viven en la carpeta `uploads/` cuando `STORAGE_PROVIDER=local`: **la copia de seguridad debe incluir esa carpeta** además de la base.

2.7 Comandos de todos los días

Comando	Qué hace
`pnpm dev`	API y web en modo desarrollo
`pnpm build`	Compila los tres paquetes
`pnpm lint` / `pnpm typecheck`	ESLint + Prettier + regla de «sin colores escritos a mano» / TypeScript estricto
`pnpm test`	Pruebas unitarias (`shared`) y de API (Supertest contra `ifts2_test`)
`pnpm e2e --build`	Compila, levanta todo y corre la suite Cypress completa a 390 y 1440 px
`pnpm verify`	Verifica los flujos de negocio y la seguridad contra el sistema real
`pnpm load` / `pnpm lighthouse`	Prueba de carga del sitio público / Lighthouse móvil
`pnpm docs:pdf`	Regenera el PDF único de documentación (`docs/entrega/`)
`pnpm package`	Genera el `.zip` de la entrega en `dist-entrega/`

2.8 Problemas frecuentes al instalar

Síntoma	Causa y solución
`pnpm db:up`: «container name "/ifts2-db" is already in use»	Ya existe un contenedor `ifts2-db` (de este u otro proyecto). Reutilizalo: `docker start ifts2-db` y, si querés una base propia, `docker exec ifts2-db psql -U ifts2 -c "CREATE DATABASE ifts2_mia"` con su `DATABASE_URL` en el `.env` (así se verificó la instalación en limpio). O eliminalo con `docker rm -f ifts2-db` si es tuyo y no tiene datos que quieras conservar.
`EADDRINUSE :3000` (o `:4000`)	Otro programa usa el puerto. Cambiá los puertos como se explicó en 2.3.
`ECONNREFUSED` o `P1001` hacia la base	Docker Desktop apagado o contenedor caído: `pnpm db:up`. El puerto es **5433**.

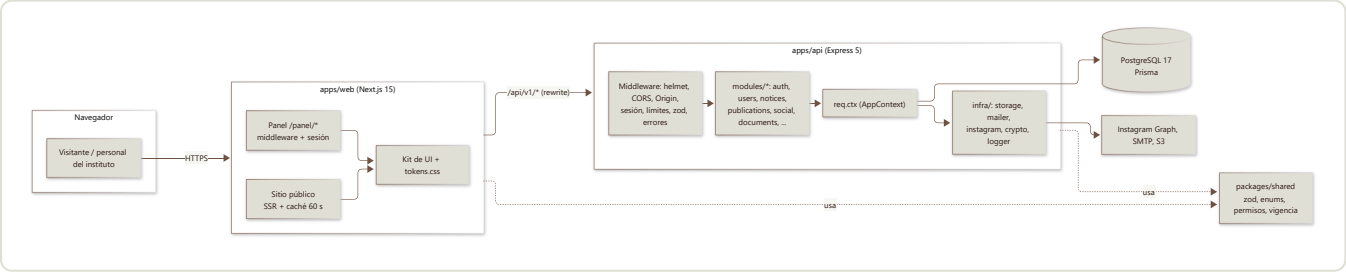
Síntoma	Causa y solución
La API no arranca y lista variables	Es a propósito: `.env` incompleto o proveedor mal configurado. Compará con `.env.example`.
El panel vuelve siempre al login	La cookie no llega: `APP_ORIGIN` tiene que ser el origen real de la web (esquema, host y puerto).
`next build` falla bajando la fuente	Necesita internet para descargar Plus Jakarta Sans de Google Fonts.
Las pruebas de API fallan por la base	Falta `ifts2_test`: `docker exec ifts2-db psql -U ifts2 -c "CREATE DATABASE ifts2_test"` (la crea sola al primer arranque del contenedor).
Cypress: `bad option: --smoke-test`	El entorno define `ELECTRON_RUN_AS_NODE`. Ejecutá con la variable sin definir: `env -u ELECTRON_RUN_AS_NODE pnpm test:e2e` (PowerShell: `$env:ELECTRON_RUN_AS_NODE=$null`).
Cypress falla con `429` al iniciar sesión	Se activó el límite de 5 intentos fallidos por IP. Reiniciá la API o usá `pnpm e2e`, que la levanta con el límite alto.
Cambié `schema.prisma` y falla el tipo de Prisma	`pnpm db:migrate:dev` (crea la migración) y `pnpm db:generate`.

3. Arquitectura del sistema

3.1 Modelo general

Arquitectura **cliente–servidor en capas, modular**, con **API REST separada** de la web:

- **Web** (Next.js 15, App Router): renderiza el sitio público en el servidor con caché (ISR de 60 s) y protege el panel con sesión.
- **API** (Express 5): reglas de negocio, validación, permisos y persistencia. Cada módulo funcional es una carpeta que se monta sola.
- **Compartido** (`packages/shared`): esquemas zod, enums, matriz de permisos y reglas de vigencia, usados **idénticos** por la API y la web.
- **Base de datos**: PostgreSQL 17 con Prisma como ORM y migraciones versionadas.
- **Servicios externos detrás de interfaces** (almacenamiento, correo, Instagram): cambiar de proveedor es configuración, no código.



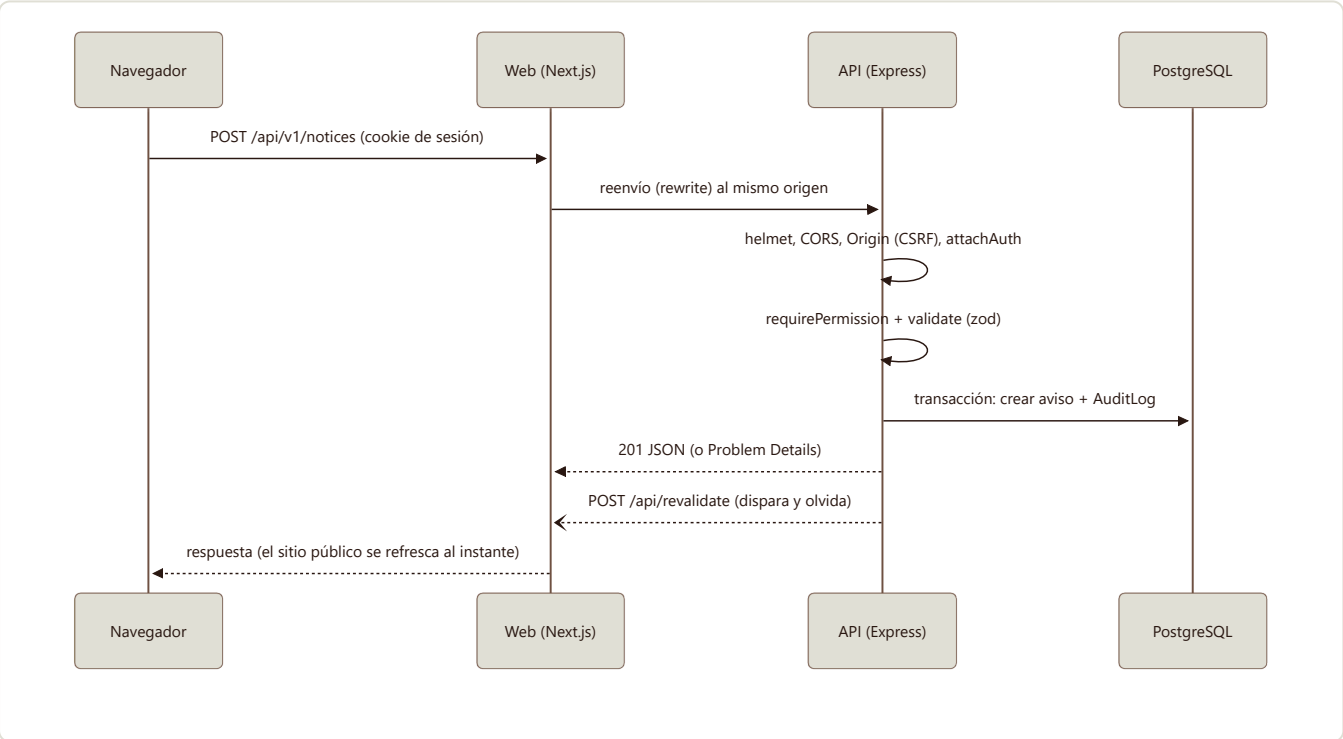
3.2 Tecnologías

Pieza	Tecnología (versión del proyecto)	Por qué
Monorepo	pnpm workspaces (pnpm 11.8)	Un repositorio para web, API y código compartido
Web	Next.js 15.5, React 19, Tailwind 3.4, Radix UI	SEO y caché para el público; Radix da accesibilidad (foco, teclado)

Pieza	Tecnología (versión del proyecto)	Por qué
API	Node 24, Express 5, TypeScript 5.9 estricto	API REST simple de testear
Base de datos	PostgreSQL 17, Prisma 6	Migraciones versionadas y tipos generados
Validación	zod 3 (en `packages/shared`)	Una sola fuente de reglas para API y web, con mensajes en español
Seguridad	argon2id (`@node-rs/argon2`), helmet, express-rate-limit, AES-256-GCM	Claves, cabeceras, límites de frecuencia y cifrado de secretos
Archivos	multer, sharp, `@aws-sdk/client-s3`	Subida validada por firma, WebP, almacenamiento local o S3
Correo	nodemailer	SMTP configurable
Registro	pino	Logs estructurados sin secretos
Pruebas	Vitest 5, Supertest 7, Cypress 16 + axe, autocannon, Lighthouse	Unitarias, API, extremo a extremo con accesibilidad, carga y rendimiento
Calidad	ESLint 9, Prettier 3, GitHub Actions	Todo se verifica en cada pull request

Las decisiones están justificadas en `docs/specs/02-decisiones-tecnicas.md` y en los ADR de `docs/decisiones/` (integraciones configurables, theming por variables CSS, producto genérico, fundación técnica, sincronización de redes).

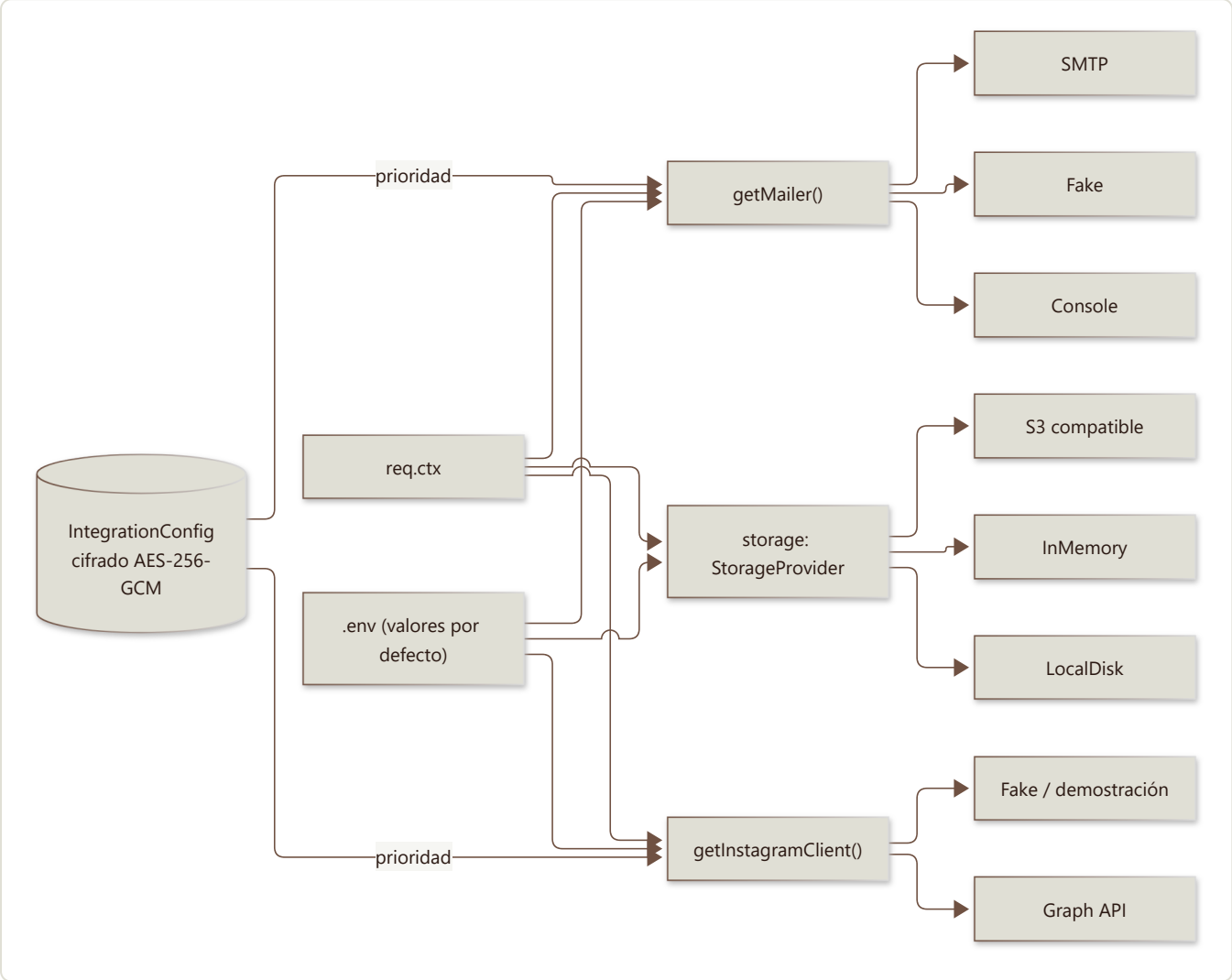
3.3 Cómo se arma un pedido



1. El navegador llama a `/api/v1/...` en el origen de la web; Next lo reenvía a la API, así la cookie de sesión es de primer nivel y no hace falta CORS entre dominios.
2. La API aplica `helmet`, CORS (solo `APP_ORIGIN`), límite de tamaño, verificación de `Origin` en métodos que modifican datos, y `attachAuth` (cookie → sesión en base, comparando el **hash** del token).

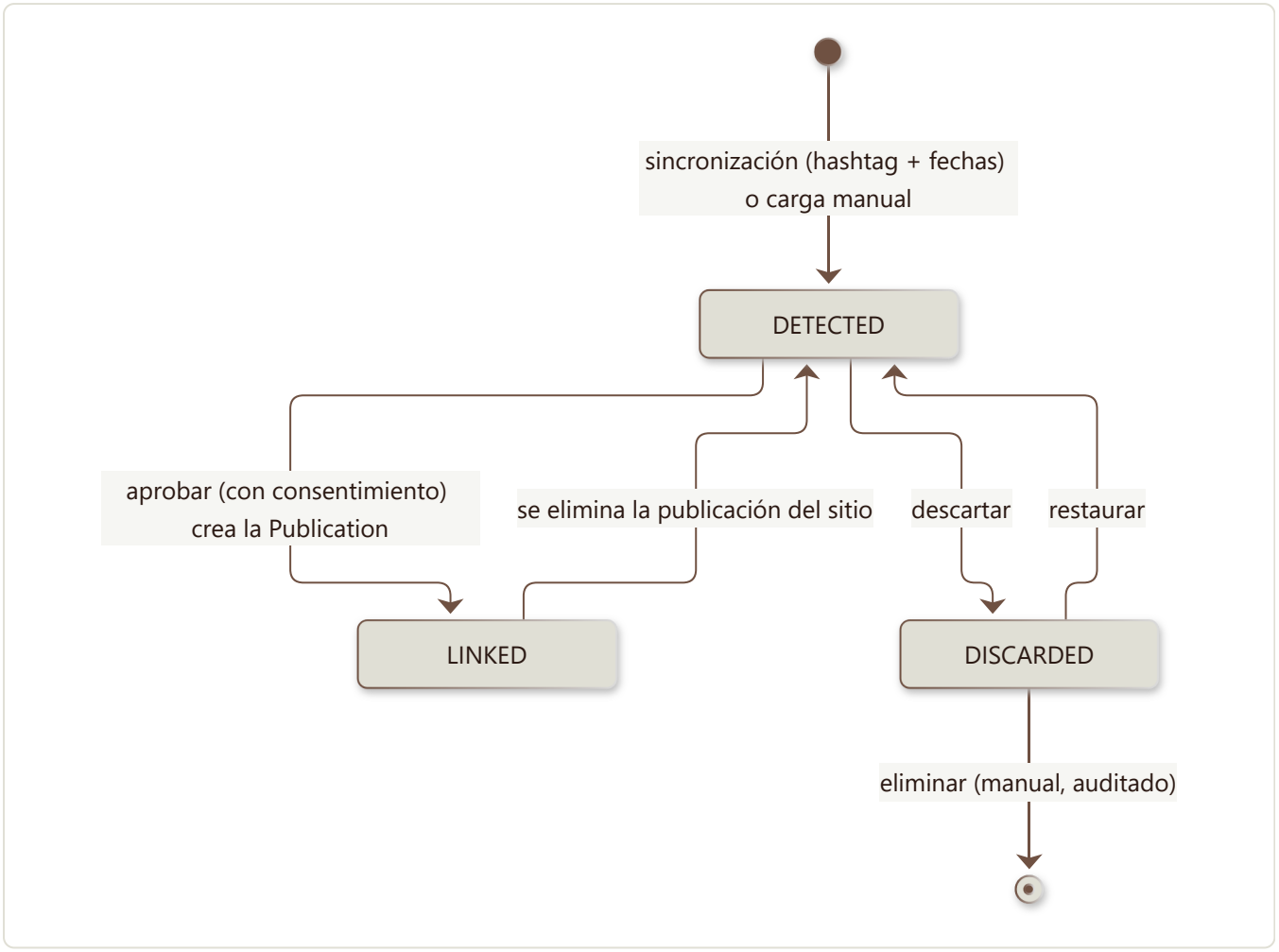
3. El módulo valida con zod (`validate`), exige permiso (`requirePermission`), usa servicios solo desde `req.ctx` , **audita** y responde. Cualquier error sale como Problem Details.
4. Tras cada modificación exitosa del panel, la API avisa a la web (`POST {APP_ORIGIN}/api/revalidate` con `REVALIDATE_SECRET`) para invalidar la caché del sitio público. Si la web no responde, no pasa nada: el sitio se actualiza solo a los 60 s.

3.4 Servicios con interfaz y modo demostración



- **Instagram y correo los configura el cliente desde el panel** (se guardan cifrados en la tabla `IntegrationConfig`); el `.env` es solo un valor por defecto. **El almacenamiento** se elige por entorno.
- Los servicios se resuelven **en cada uso** (no se cachean), para que un cambio en el panel aplique al instante.
- Sin configuración real, todo funciona en **modo demostración** (Instagram falso, correo por consola) y `GET /health` lo informa con `demoMode: true` ; el panel muestra un aviso.

3.5 Flujo de moderación de Instagram



4. Estructura del proyecto

```
ifts2-plataforma/
├── apps/
│   ├── api/                 API REST (Express 5 + TypeScript)
│   │   ├── src/
│   │   │   ├── app.ts       createApp(): middlewares globales y montaje de módulos
│   │   │   ├── server.ts    arranque; valida la configuración antes de abrir el puerto
│   │   │   ├── config.ts    variables de entorno validadas con zod
│   │   │   ├── context.ts   AppContext (req.ctx): prisma, storage, getMailer(), getInstagramClient()
│   │   │   ├── routes.ts    autodescubrimiento: monta src/modules/*/index.ts
│   │   │   ├── middleware/  auth, permisos, validate, origin (CSRF), rate-limit, errores, upload,
revalidate-web
│   │   │   ├── infra/       storage/, mailer/, instagram/, crypto, logger, errors, integration-store
│   │   │   ├── lib/         password (argon2id), pagination, markdown, settings, documents, cover
│   │   │   └── modules/     un directorio por módulo funcional (cada uno con index.ts y README)
│   │   └── test/            unit/, api/ (Supertest) y helpers/
│   └── web/                 Next.js 15 (App Router)
│       ├── src/app/         rutas: (public)/, panel/, api/revalidate, ui-kit
│       ├── src/components/  ui/ (kit accesible), layout/ (PublicLayout, PanelShell)
│       ├── src/features/    public/, panel-a/ (login, resumen, publicaciones, avisos, mensajes),
│       │                    panel-b/ (horarios, archivo, configuración)
│       ├── src/lib/         api (cliente), session, site, dates
│       ├── src/styles/      tokens.css (generado: paleta y tipografías)
│       └── cypress/         e2e/ (pruebas de punta a punta), support/, fixtures/
├── packages/shared/         zod, enums, permisos, vigencia, zona horaria, guía de Instagram
├── prisma/                  schema.prisma, migrations/, seed.ts (demo/prod), seed-data.ts, seed-plan.ts
├── scripts/                 e2e, verify-flows, load/, tokens, contrast, docs/ (PDF y DER), package
├── docs/                    specs/, design/ (Figma), decisiones/ (ADR), capturas/, evidencias/, entrega/
├── docker-compose.yml       contenedor ifts2-db (PostgreSQL 17)
└── .env.example             todas las variables, comentadas
```

Sincronización incremental (ADR-038). Para no pedir a Instagram todo el listado en cada consulta, la API guarda (cifrado, en `IntegrationConfig`, clave `instagram_sync`) el registro de cada fetch exitoso: fecha, cantidad, **id de la cuenta**, hashtag y método de conexión, sin secretos. La siguiente sincronización trae solo lo posterior al último fetch (con un margen de 6 horas; la idempotencia por `externalId` evita duplicados). Un fetch fallido no avanza la marca. La carga es **completa** la primera vez, al forzarla (`POST /social/sync?full=true`, botón «Sincronización completa» del panel) y cuando cambia la cuenta, el hashtag o el método de conexión.

4.1 Módulos de la API (`apps/api/src/modules/`)

Cada carpeta exporta `{ mountPath, router }` y se monta sola bajo `/api/v1`; agregar un módulo **no requiere editar archivos compartidos**.

Módulo	Responsabilidad
`health`	Estado de la API, la base y las integraciones (sin secretos)
`auth`	Login/logout, sesión actual, cambio de clave, límite de intentos
`users`	Alta, edición de rol, desactivar/reactivar, restablecer clave (clave temporal que se muestra una vez)
`audit`	Registro y consulta de auditoría (`audit.fromReq`)
`integrations`	Configuración de Instagram (OAuth, token, probar, desconectar) y correo; todo cifrado
`files`	Descarga de archivos locales con URL firmada de corta duración
`categories`	Categorías (únicas por nombre y tipo; borrar → 409 si están en uso)
`publications`	Publicaciones del sitio: estados, vigencia, portada, eliminación manual

Módulo	Responsabilidad
`social`	Sincronización de Instagram, moderación (aprobar/descartar/restaurar), carga manual, cron
`dashboard`	Datos del Resumen del panel
`notices`	Avisos académicos (CRUD, pausar/reanudar, vencimiento calculado)
`terms`	Ciclos lectivos y **cambio del ciclo público** (uno solo)
`schedules`	Horarios por comisión con grilla adjunta
`exam-calls`	Llamados a examen / mesas
`documents`	Archivo documental: subir, archivar, restaurar, eliminar definitivamente, detalle con referencias
`settings`	Ajustes del sitio (sede, contactos, hashtag)
`content`	Texto editable de Institucional, La carrera e Ingreso (Markdown acotado)
`links`	Enlaces de interés (SIU Guaraní, Aulas Virtuales, etc.)
`contact`	Formulario público de contacto y bandeja de mensajes del panel
`public`	Lectura pública con la vigencia ya aplicada (`/public/*`), con caché HTTP corta

La referencia completa de endpoints, con permisos y códigos de estado, está en `docs/specs/04-api.md`; cada módulo tiene su `README.md`.

4.2 Permisos por rol

Hay **cuatro roles de sistema** (SUPERADMIN, ADMIN, BEDELIA y EDITOR). Los permisos son un **catálogo fijo** (`ACTIONS` en `packages/shared/src/permissions.ts`, por ejemplo `notices:manage`) y los **roles son datos** (tablas `Role` y `RolePermission`): el Superadministrador crea, edita y elimina roles y tilda sus permisos en **Panel → Roles y permisos** (RF11, ADR-031). La API (`requirePermission`) y la web (para ocultar opciones) leen los permisos de la sesión en cada pedido, así que un cambio rige sin volver a ingresar. SUPERADMIN siempre tiene todo y no se edita ni se elimina; ADMIN, BEDELIA y EDITOR se pueden editar y eliminar (409 si tienen usuarios). El último SUPERADMIN activo no se puede desactivar ni cambiar de rol (409). Las rutas `/integrations/*` exigen el permiso `integrations:manage`, no un rol.

La semilla solo crea el SUPERADMIN inicial en ausencia de usuarios y solo acepta una cuenta existente si es SUPERADMIN activa. No restablece claves, reactiva cuentas ni eleva privilegios existentes. Si falta acceso administrativo, consultar el procedimiento explícito y auditado de recuperación.

La tabla siguiente es la **matriz inicial** que carga la semilla (después manda lo que el Superadministrador haya tildado). SUPERADMIN tiene todo y no figura.

Recurso	ADMIN	BEDELIA	EDITOR
Resumen	✓	✓	✓
Publicaciones y redes (moderar, crear, pausar)	✓		✓
Avisos, horarios, llamados, archivo, mensajes (ver)	✓	✓	
Ciclo público, ajustes, usuarios, integraciones, auditoría	✓		
Eliminar mensajes de contacto	✓		

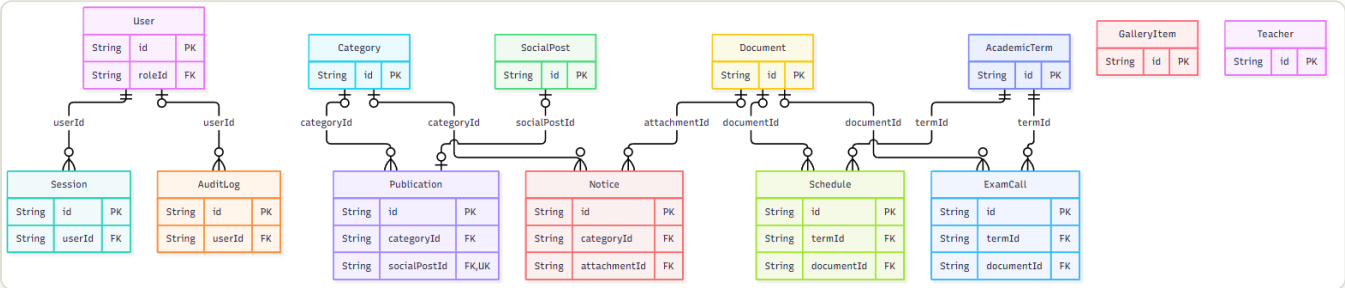
5. Base de datos

Motor: **PostgreSQL 17**. Acceso: **Prisma 6**. La fuente de verdad es `prisma/schema.prisma`; las migraciones (`prisma/migrations/`) son los scripts SQL de creación. Reglas del modelo: (1) nada se borra solo: se archiva o se pausa; (2) la vigencia se calcula al consultar; (3) todo cambio de contenido genera una fila en `AuditLog`.

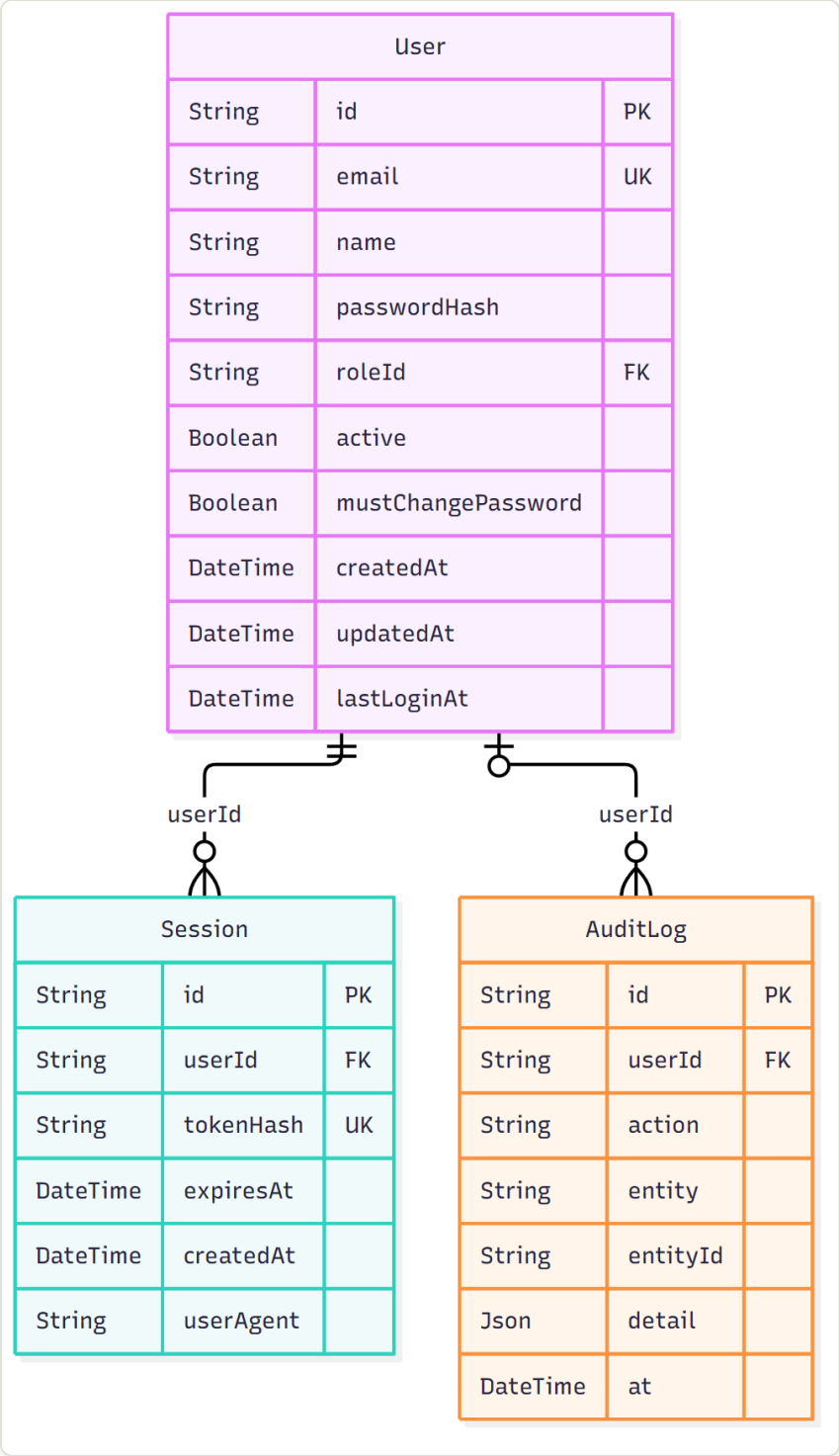
5.1 Diagrama Entidad–Relación (DER)

Los diagramas se generan **desde el schema** con `node scripts/docs/prisma-to-docs.mjs` (fuentes `.mmd` en `docs/entrega/diagramas/`) y se renderizan con `@mermaid-js/mermaid-cli`. Por legibilidad se muestra una vista general (solo claves y relaciones) y un diagrama por área con todos los campos. `SiteSetting`, `ContentSection`, `ExternalLink`, `ContactMessage`, `InstagramToken` e `IntegrationConfig` no tienen relaciones con otras tablas.

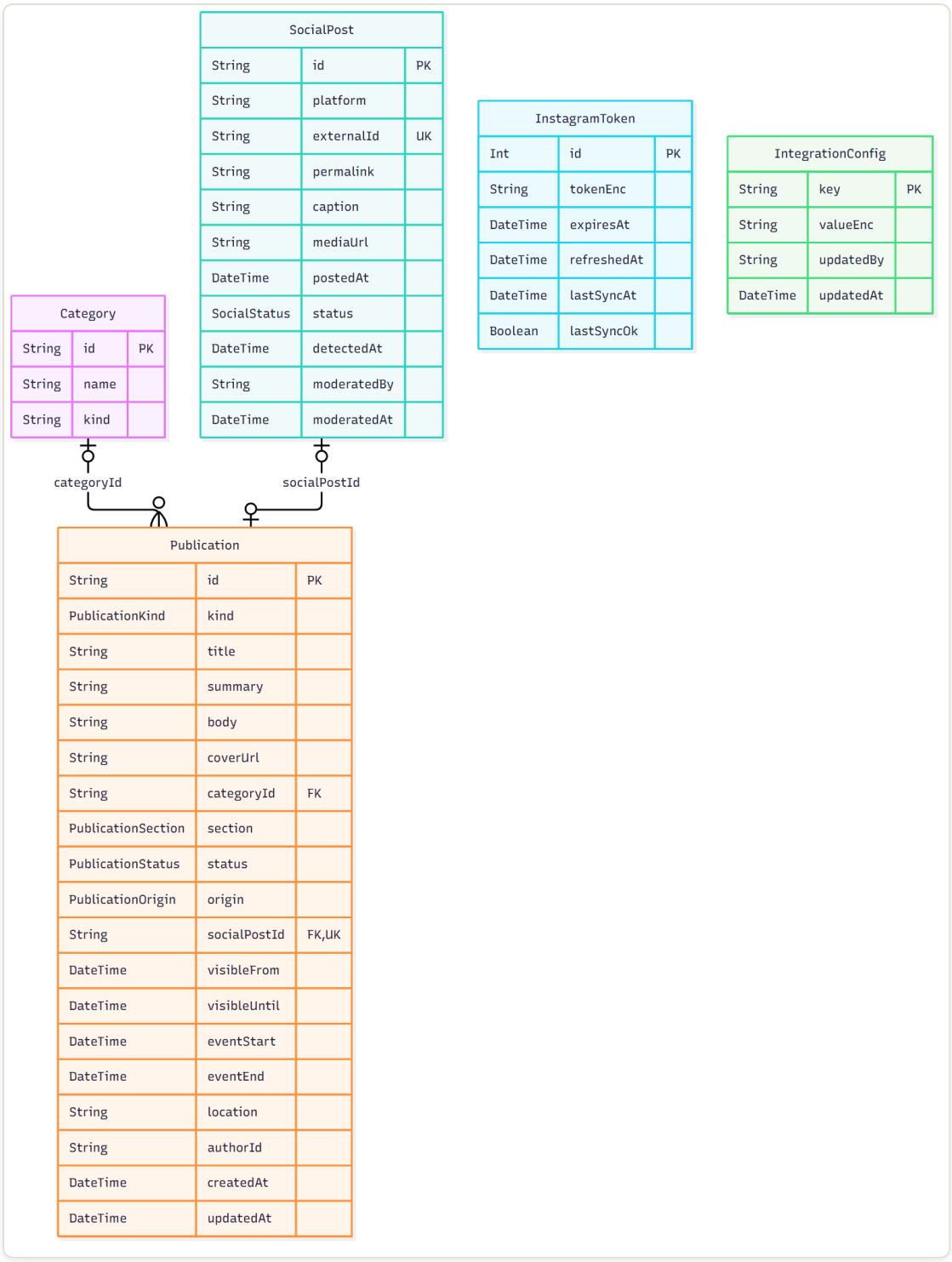
Vista general (claves y relaciones):



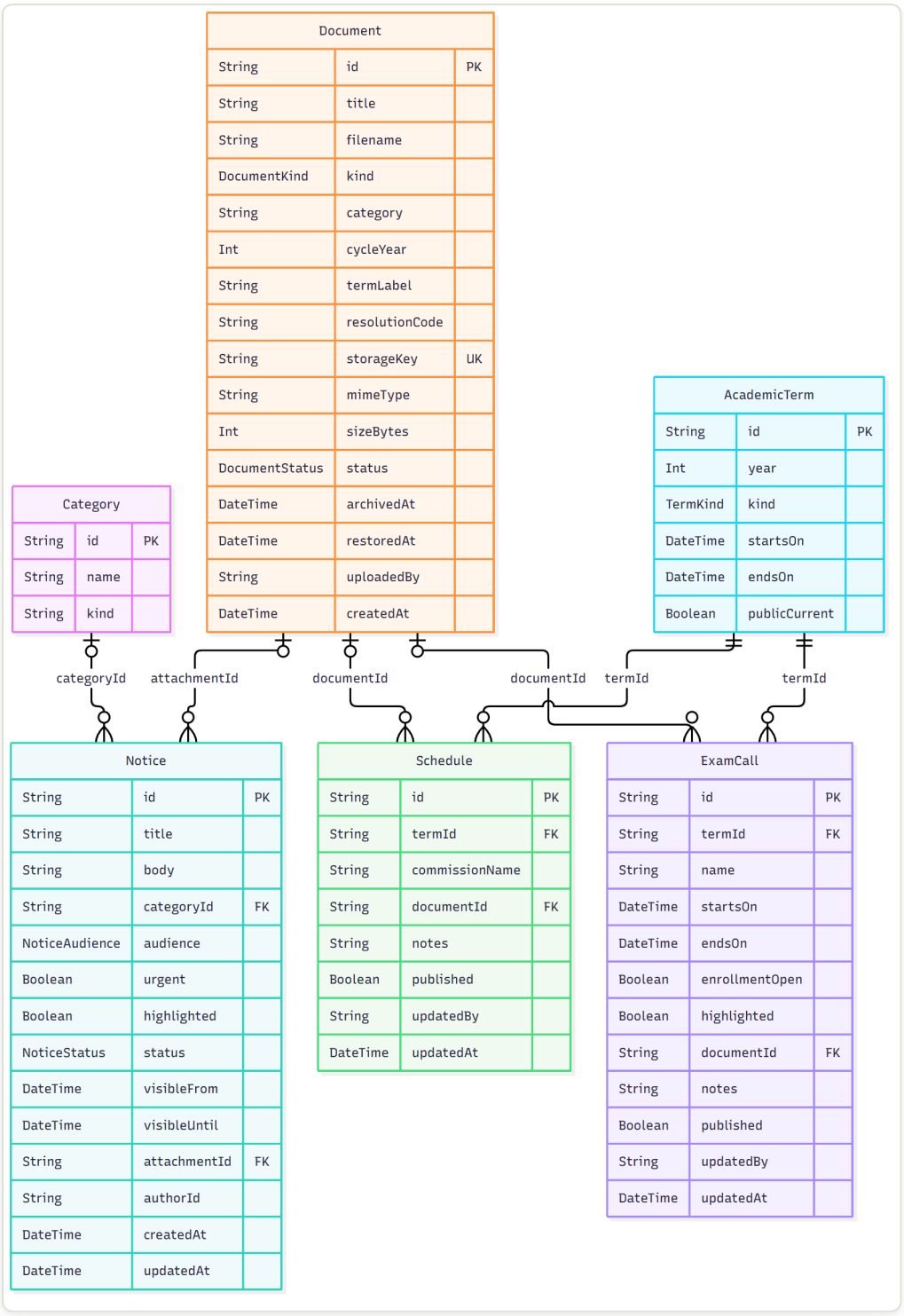
Acceso y auditoría:



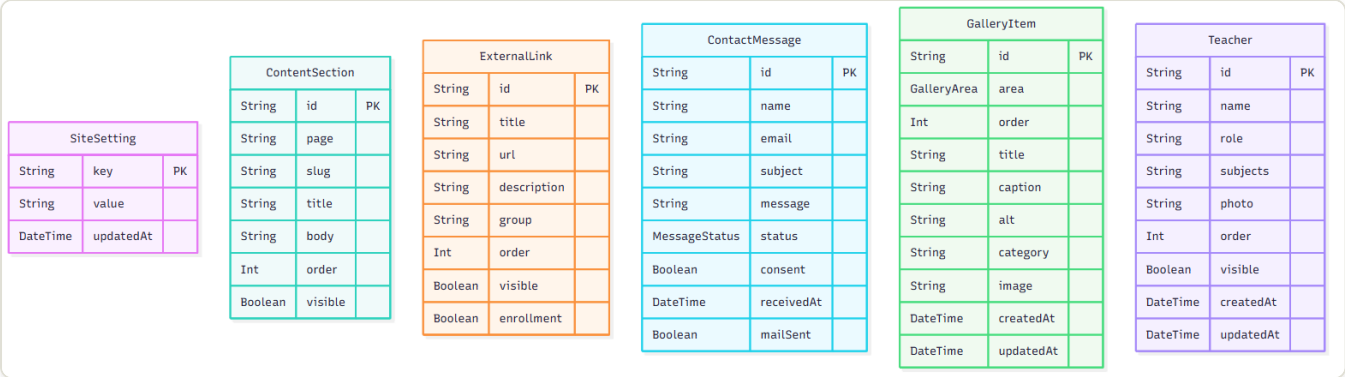
Publicaciones, redes e integraciones:



Bloque académico (avisos, ciclos, horarios, llamados y archivo):



Contenido del sitio y contacto:



5.2 Tablas, campos y relaciones

Tablas y campos

> Esta sección se genera con `node scripts/docs/prisma-to-docs.mjs` a partir de `prisma/schema.prisma`; no se edita a mano. Los tipos son los de Prisma (`String` = texto, `Int` = entero, `Boolean` = verdadero/falso, `DateTime` = fecha y hora en UTC, `Json` = objeto JSON); en PostgreSQL se materializan como `TEXT`, `INTEGER`, `BOOLEAN`, `TIMESTAMP(3)` y `JSONB`. Las filas «Relación con …» son los lados de navegación de Prisma y no son columnas.

Role

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`key`	String	único, obligatorio	Clave interna única del rol (SUPERADMIN, ADMIN, BEDELIA, EDITOR o la de un rol creado).
`name`	String	único, obligatorio	Nombre que se muestra en el panel. No se repite entre roles.
`description`	String	opcional	Explicación breve de para qué sirve el rol.
`isSystem`	Boolean	obligatorio, por defecto false	Verdadero para los roles que crea la semilla. SUPERADMIN no se edita ni se elimina; los demás sí se pueden eliminar si no tienen usuarios.
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).
`permissions`	RolePermission[]	(no es columna)	Permisos del rol (relación inversa con RolePermission).
`users`	User[]	(no es columna)	Usuarios que tienen este rol (relación inversa).

RolePermission

Campo	Tipo	Restricciones	Significado
`roleId`	String	obligatorio, FK → Role.id (Cascade)	Rol al que pertenece el permiso. Al borrar el rol se borran sus filas.
`role`	Role	(no es columna)	Relación con Role.

Campo	Tipo	Restricciones	Significado
`permission`	String	obligatorio	Permiso del catálogo fijo en el código (por ejemplo notices:manage). No se repite dentro del rol.

Restricciones e índices de la tabla: `@@id([roleId, permission])`.

User

Personas del instituto que usan el panel, con su rol.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`email`	String	único, obligatorio	Correo con el que inicia sesión. No se repite entre usuarios.
`name`	String	obligatorio	Nombre y apellido que se muestra en el panel y en la auditoría.
`passwordHash`	String	obligatorio	Clave guardada como hash argon2id. Nunca se guarda ni se devuelve la clave en claro.
`roleId`	String	obligatorio, FK → Role.id	Rol del usuario (relación con Role). Sus permisos se leen en cada pedido.
`role`	Role	(no es columna)	Relación con Role.
`active`	Boolean	obligatorio, por defecto true	Si es falso, la persona no puede iniciar sesión (se desactiva en lugar de borrar, para conservar la auditoría).
`mustChangePassword`	Boolean	obligatorio, por defecto false	Verdadero tras crear el usuario con clave temporal o restablecerla: obliga a cambiarla en el próximo ingreso.
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de la última modificación (la actualiza Prisma sola).
`lastLoginAt`	DateTime	opcional	Último inicio de sesión exitoso.
`sessions`	Session[]	(no es columna)	Sesiones abiertas del usuario (relación inversa).
`audit`	AuditLog[]	(no es columna)	Acciones auditadas realizadas por el usuario (relación inversa).

Restricciones e índices de la tabla: `@@index([roleId])`.

Session

Sesiones de inicio de sesión (revocables).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`userId`	String	obligatorio, FK → User.id (Cascade)	Usuario dueño de la sesión. Al borrar el usuario se borran sus sesiones.
`user`	User	(no es columna)	Relación con User.

Campo	Tipo	Restricciones	Significado
`tokenHash`	String	único, obligatorio	Hash del token de la cookie de sesión (nunca el token en claro). Permite revocar sesiones.
`expiresAt`	DateTime	obligatorio	Momento en que la sesión deja de ser válida (8 horas, o 14 días con «recordarme»).
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`userAgent`	String	opcional	Navegador desde el que se inició la sesión (informativo).

Restricciones e índices de la tabla: `@@index([userId])`, `@@index([expiresAt])`.

Category

Categorías para publicaciones, avisos y galería.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`name`	String	obligatorio	Nombre de la categoría (por ejemplo «Inscripciones»).
`kind`	String	obligatorio	A qué se aplica: PUBLICATION, NOTICE o GALLERY. El par (nombre, tipo) no se repite.
`publications`	Publication[]	(no es columna)	Publicaciones de la categoría (relación inversa).
`notices`	Notice[]	(no es columna)	Avisos de la categoría (relación inversa).

Restricciones e índices de la tabla: `@@unique([name, kind])`.

SocialPost

Publicaciones detectadas en Instagram, a la espera de moderación humana.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`platform`	String	obligatorio, por defecto "INSTAGRAM"	Red social de origen (hoy INSTAGRAM).
`externalId`	String	único, obligatorio	Identificador del medio en Instagram. Es único: evita importar dos veces la misma publicación.
`permalink`	String	obligatorio	Enlace público a la publicación original en la red.
`caption`	String	obligatorio	Texto (leyenda) de la publicación original.
`mediaUrl`	String	obligatorio	Dirección temporal de la imagen; al aprobar se copia al almacenamiento propio.
`postedAt`	DateTime	obligatorio	Fecha en que se publicó en la red.

Campo	Tipo	Restricciones	Significado
`status`	SocialStatus	obligatorio, por defecto DETECTED	DETECTED (pendiente de moderar), LINKED (aprobada, ya tiene Publication) o DISCARDED (descartada).
`detectedAt`	DateTime	obligatorio, por defecto now()	Cuándo la detectó la sincronización.
`moderatedBy`	String	opcional	Id del usuario que la aprobó o descartó.
`moderatedAt`	DateTime	opcional	Cuándo se moderó.
`publication`	Publication	(no es columna)	Publicación del sitio creada a partir de esta (relación inversa 1 a 1).

Restricciones e índices de la tabla: `@@index([status, postedAt])`.

Publication

Noticias y eventos del sitio (manuales o aprobadas desde Instagram).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`kind`	PublicationKind	obligatorio, por defecto NEWS	NEWS (noticia) o EVENT (evento).
`title`	String	obligatorio	Título que ve el visitante.
`summary`	String	obligatorio	Resumen corto para tarjetas y listados.
`body`	String	obligatorio, por defecto ""	Texto completo (Markdown acotado, se sanitiza al mostrarlo).
`coverUrl`	String	opcional	Imagen de portada: una dirección http(s) o una clave del almacenamiento propio.
`categoryId`	String	opcional, FK → Category.id	Categoría opcional.
`category`	Category	(no es columna)	Relación con Category.
`section`	PublicationSection	obligatorio, por defecto BOTH	Dónde se muestra: HOME (inicio), NEWS (novedades) o BOTH.
`status`	PublicationStatus	obligatorio, por defecto DRAFT	DRAFT (borrador), SCHEDULED (programada), PUBLISHED (publicada) o PAUSED (pausada). La vigencia real se calcula al consultar.
`origin`	PublicationOrigin	obligatorio, por defecto MANUAL	MANUAL (cargada en el panel) o INSTAGRAM (aprobada desde una publicación de la red).
`socialPostId`	String	único, opcional, FK → SocialPost.id	Publicación de Instagram de la que proviene. Única: una publicación de red genera, como máximo, una del sitio.
`socialPost`	SocialPost	(no es columna)	Relación con SocialPost.
`visibleFrom`	DateTime	opcional	Desde cuándo es visible (inclusivo). Vacío = sin límite.

Campo	Tipo	Restricciones	Significado
`visibleUntil`	DateTime	opcional	Hasta cuándo es visible (exclusivo). Vacío = sin vencimiento.
`eventStart`	DateTime	opcional	Inicio del evento (solo si kind = EVENT).
`eventEnd`	DateTime	opcional	Fin del evento.
`location`	String	opcional	Lugar del evento.
`authorId`	String	obligatorio	Id del usuario autor (se guarda como texto, sin clave foránea, para no perder el dato si se desactiva).
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([status, visibleFrom, visibleUntil])`, `@@index([categoryId])`.

Notice

Avisos académicos / cartelera oficial.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`title`	String	obligatorio	Título del aviso.
`body`	String	obligatorio	Texto del aviso.
`categoryId`	String	opcional, FK → Category.id	Categoría opcional (tipo NOTICE).
`category`	Category	(no es columna)	Relación con Category.
`audience`	NoticeAudience	obligatorio, por defecto ALL	A quién va dirigido: ALL, STUDENTS, ASPIRANTS o GRADUATES.
`urgent`	Boolean	obligatorio, por defecto false	Marca el aviso como urgente (se muestra destacado).
`highlighted`	Boolean	obligatorio, por defecto false	Destaca un evento próximo.
`status`	NoticeStatus	obligatorio, por defecto ACTIVE	ACTIVE o PAUSED. «Vencido» no se guarda: se calcula con visibleUntil.
`visibleFrom`	DateTime	obligatorio, por defecto now()	Desde cuándo es visible (por defecto, ahora).
`visibleUntil`	DateTime	opcional	Hasta cuándo es visible. Vacío = sin vencimiento.
`attachmentId`	String	opcional, FK → Document.id	Documento adjunto opcional.
`attachment`	Document	(no es columna)	Relación con Document.
`authorId`	String	obligatorio	Id del usuario que lo creó.
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.

Campo	Tipo	Restricciones	Significado
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([status, visibleFrom, visibleUntil])`.

Document

Repositorio documental: archivos subidos, archivados o restaurados, y adjuntos.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`title`	String	obligatorio	Nombre que se muestra del documento.
`filename`	String	obligatorio	Nombre original del archivo subido.
`kind`	DocumentKind	obligatorio, por defecto OTHER	Tipo: SCHEDULE, EXAM_CALENDAR, RESOLUTION, PLAN, NOTICE_ATTACHMENT u OTHER.
`category`	String	opcional	Etiqueta libre para la tabla del archivo (por ejemplo «Mesas Examen»).
`cycleYear`	Int	opcional	Ciclo lectivo (año) al que corresponde.
`termLabel`	String	opcional	Cuatrimestre en texto (por ejemplo «2.° Cuat.»).
`resolutionCode`	String	opcional	Código de la resolución, si es una resolución.
`storageKey`	String	único, obligatorio	Clave del archivo en el almacenamiento (única). No sale de la API: se descarga con URL firmada.
`mimeType`	String	obligatorio	Tipo de archivo (por ejemplo application/pdf).
`sizeBytes`	Int	obligatorio	Tamaño en bytes.
`status`	DocumentStatus	obligatorio, por defecto ACTIVE	ACTIVE o ARCHIVED (archivado: sale del sitio pero no se borra).
`archivedAt`	DateTime	opcional	Cuándo se archivó.
`restoredAt`	DateTime	opcional	Cuándo se restauró por última vez.
`uploadedBy`	String	obligatorio	Id del usuario que lo subió.
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`notices`	Notice[]	(no es columna)	Avisos que lo adjuntan (relación inversa).
`schedules`	Schedule[]	(no es columna)	Horarios que lo usan (relación inversa).
`examCalls`	ExamCall[]	(no es columna)	Llamados que lo usan (relación inversa).

Restricciones e índices de la tabla: `@@index([cycleYear, status])`.

AcademicTerm

Ciclos lectivos; uno solo es el «público» y gobierna qué horarios y llamados se ven.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`year`	Int	obligatorio	Año del ciclo lectivo.
`kind`	TermKind	obligatorio	FIRST (1.er cuatrimestre), SECOND (2.º) o EXTRAORDINARY. El par (año, tipo) no se repite.
`startsOn`	DateTime	opcional	Inicio del ciclo.
`endsOn`	DateTime	opcional	Fin del ciclo.
`publicCurrent`	Boolean	obligatorio, por defecto false	Verdadero en el ciclo que muestra el sitio. Solo uno puede serlo (índice único parcial en la base).
`schedules`	Schedule[]	(no es columna)	Horarios del ciclo (relación inversa).
`examCalls`	ExamCall[]	(no es columna)	Llamados del ciclo (relación inversa).

Restricciones e índices de la tabla: `@@unique([year, kind])`.

Schedule

Horarios de cursada por comisión (con grilla adjunta).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`termId`	String	obligatorio, FK → AcademicTerm.id	Ciclo al que pertenece.
`term`	AcademicTerm	(no es columna)	Relación con AcademicTerm.
`commissionName`	String	obligatorio	Comisión (por ejemplo «Comisión A y B»).
`documentId`	String	opcional, FK → Document.id (Restrict)	Grilla de horarios adjunta (documento).
`document`	Document	(no es columna)	Relación con Document.
`notes`	String	opcional	Observaciones.
`published`	Boolean	obligatorio, por defecto false	Si está publicado. Solo se ve si además su ciclo es el público.
`updatedBy`	String	obligatorio	Id del usuario que lo modificó por última vez.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([termId, published])`.

ExamCall

Llamados a examen / mesas.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`termId`	String	obligatorio, FK → AcademicTerm.id	Ciclo al que pertenece.
`term`	AcademicTerm	(no es columna)	Relación con AcademicTerm.
`name`	String	obligatorio	Nombre del llamado (por ejemplo «Turno extraordinario de Diciembre 2026»).
`startsOn`	DateTime	obligatorio	Primer día del llamado.
`endsOn`	DateTime	obligatorio	Último día del llamado.
`enrollmentOpen`	Boolean	obligatorio, por defecto false	Si la inscripción a las mesas está abierta.
`highlighted`	Boolean	obligatorio, por defecto false	Destaca el llamado próximo.
`documentId`	String	opcional, FK → Document.id	Calendario o documento adjunto.
`document`	Document	(no es columna)	Relación con Document.
`notes`	String	opcional	Observaciones.
`published`	Boolean	obligatorio, por defecto false	Si está publicado. Solo se ve si además su ciclo es el público.
`updatedBy`	String	obligatorio	Id del usuario que lo modificó por última vez.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([termId, published])`.

SiteSetting

Ajustes clave/valor del sitio (sede, contactos, hashtag, etc.).

Campo	Tipo	Restricciones	Significado
`key`	String	PK, obligatorio	Nombre del ajuste (por ejemplo institution_name, address, instagram_hashtag).
`value`	String	obligatorio	Valor del ajuste, siempre texto.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

ContentSection

Texto editable de las páginas estáticas (Institucional, La carrera, Ingreso).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`page`	String	obligatorio	Página a la que pertenece: institucional, carrera, estudiantes, ingreso o inscripciones.
`slug`	String	obligatorio	Identificador corto de la sección dentro de la página. (página, slug) no se repite.

Campo	Tipo	Restricciones	Significado
`title`	String	obligatorio	Título de la sección.
`body`	String	obligatorio	Texto en Markdown acotado, sanitizado al mostrarlo.
`order`	Int	obligatorio, por defecto 0	Posición dentro de la página.
`visible`	Boolean	obligatorio, por defecto true	Si se muestra en el sitio.
`kind`	String	obligatorio, por defecto "TEXT"	Tipo de sección: TEXT (título y texto) o GALLERY (muestra una galería).
`galleryId`	String	opcional, FK → Gallery.id (SetNull)	Galería que se muestra cuando la sección es de tipo GALLERY.
`gallery`	Gallery	(no es columna)	[completar]

Restricciones e índices de la tabla: `@@unique([page, slug])`, `@@index([galleryId])`.

ExternalLink

Enlaces de interés (SIU Guaraní, Aulas Virtuales, etc.).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`title`	String	obligatorio	Texto del enlace (por ejemplo «SIU Guaraní»).
`url`	String	obligatorio	Dirección del enlace.
`description`	String	opcional	Descripción breve opcional.
`group`	String	obligatorio, por defecto "academico"	Grupo donde se agrupa (por defecto «academico»).
`order`	Int	obligatorio, por defecto 0	Posición dentro del grupo.
`visible`	Boolean	obligatorio, por defecto true	Si se muestra en el sitio.
`showHome`	Boolean	obligatorio, por defecto true	[completar]
`showStudents`	Boolean	obligatorio, por defecto true	[completar]
`enrollment`	Boolean	obligatorio, por defecto false	Si es el enlace donde se inscriben los alumnos (el único que muestra la página Inscripciones; A07).

ContactMessage

Consultas enviadas desde el formulario del sitio.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`name`	String	obligatorio	Nombre de quien escribe.

Campo	Tipo	Restricciones	Significado
`email`	String	obligatorio	Correo de respuesta.
`subject`	String	obligatorio	Asunto.
`message`	String	obligatorio	Texto de la consulta.
`status`	MessageStatus	obligatorio, por defecto NEW	NEW, READ o ANSWERED.
`consent`	Boolean	obligatorio	Aceptó el aviso de uso de datos (Ley 25.326). Es obligatorio.
`receivedAt`	DateTime	obligatorio, por defecto now()	Cuándo llegó.
`mailSent`	Boolean	obligatorio, por defecto false	Si el correo de aviso salió. El mensaje se guarda siempre, aunque el correo falle.

Gallery

Galerías de fotos con nombre; se agregan a cualquier página como sección.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`name`	String	obligatorio	Nombre de la galería; se ve como título de la sección donde se muestra.
`order`	Int	obligatorio, por defecto 0	Posición de la galería en la lista del panel.
`items`	GalleryItem[]	(no es columna)	[completar]
`sections`	ContentSection[]	(no es columna)	[completar]
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

GalleryItem

Fotos de una galería, con título, descripción y categoría.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`galleryId`	String	obligatorio, FK → Gallery.id (Cascade)	Galería a la que pertenece la foto (se borra junto con ella).
`gallery`	Gallery	(no es columna)	[completar]
`order`	Int	obligatorio, por defecto 0	Posición dentro de la galería (de menor a mayor).
`title`	String	obligatorio, por defecto ""	Título de la diapositiva (opcional).
`caption`	String	obligatorio, por defecto ""	Descripción visible debajo de la imagen (opcional).
`alt`	String	obligatorio, por defecto ""	Texto alternativo de la imagen para lectores de pantalla.

Campo	Tipo	Restricciones	Significado
`category`	String	obligatorio, por defecto ""	Categoría opcional (por ejemplo «Prácticas»); alimenta el filtro del carrusel de Estudiantes.
`image`	String	obligatorio	Ruta propia del sitio (/img/...) o clave del almacenamiento (gallery/.webp); nunca una URL firmada.
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([galleryId, order])`.

InstagramToken

Estado del token y de la sincronización con Instagram (una sola fila).

Campo	Tipo	Restricciones	Significado
`id`	Int	PK, obligatorio, por defecto 1	Identificador único (UUID generado por la aplicación).
`tokenEnc`	String	obligatorio	Reservado; el token en claro vive cifrado en IntegrationConfig y este campo queda vacío a propósito.
`expiresAt`	DateTime	obligatorio	Vencimiento del token de larga duración (~60 días; se renueva a los ~50).
`refreshedAt`	DateTime	obligatorio, por defecto now()	Última renovación.
`lastSyncAt`	DateTime	opcional	Última sincronización con Instagram.
`lastSyncOk`	Boolean	opcional	Si esa sincronización terminó bien.

IntegrationConfig

Configuración de integraciones cargada por el cliente desde el panel, cifrada.

Campo	Tipo	Restricciones	Significado
`key`	String	PK, obligatorio	Qué integración configura: «instagram» o «mail».
`valueEnc`	String	obligatorio	JSON de la configuración cifrado con AES-256-GCM (clave APP_ENCRYPTION_KEY). Nunca se devuelve en claro.
`updatedBy`	String	opcional	Id del usuario que la guardó por última vez.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

AuditLog

Registro de auditoría: quién hizo qué y cuándo.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`userId`	String	opcional, FK → User.id (SetNull)	Quién realizó la acción. Si el usuario se borra queda vacío (SetNull).
`user`	User	(no es columna)	Relación con User.
`action`	String	obligatorio	Acción realizada (por ejemplo notice.create, document.archive).
`entity`	String	obligatorio	Tipo de entidad afectada.
`entityId`	String	opcional	Id de la entidad afectada.
`detail`	Json	opcional	Datos adicionales en JSON (sin secretos).
`at`	DateTime	obligatorio, por defecto now()	Cuándo ocurrió.

Restricciones e índices de la tabla: `@@index([entity, entityId])`, `@@index([at])`.

Teacher

Plantel docente y materias a cargo (con baja lógica).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`name`	String	obligatorio	Nombre y apellido que se muestra en el sitio.
`role`	String	obligatorio, por defecto ""	Cargo o rol libre (por ejemplo «Profesora»); vacío si no se cargó.
`subjects`	String[]	obligatorio, por defecto []	Materias a cargo (texto libre; el plan de estudios vive en ContentSection).
`photo`	String	obligatorio, por defecto ""	Foto opcional (ruta propia o clave del almacenamiento); vacío = sin foto. Requiere consentimiento (Ley 25.326).
`order`	Int	obligatorio, por defecto 0	Posición en la lista del sitio.
`visible`	Boolean	obligatorio, por defecto true	Falso = baja lógica: no se muestra en el sitio, pero se conserva. Solo una persona oculta puede eliminarse.
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([visible, order])`.

Índice único parcial (SQL de la migración inicial)

Prisma no puede expresar este índice en el schema, por eso vive en `prisma/migrations/20261029224122_init/migration.sql`: garantiza que **solo un ciclo** tenga `publicCurrent = true`.

Enumeraciones

Enumeración	Valores	Uso
`PublicationKind`	`NEWS` (Noticia / se muestra en Novedades); `EVENT` (Evento)	Tipo de publicación.
`PublicationStatus`	`DRAFT` (Borrador); `SCHEDULED` (Programada); `PUBLISHED` (Publicada); `PAUSED` (Pausada)	Estado de una publicación.
`PublicationOrigin`	`MANUAL` (Cargada a mano en el panel); `INSTAGRAM` (Aprobada desde Instagram)	Origen de la publicación.
`PublicationSection`	`HOME` (Inicio); `NEWS` (Noticia / se muestra en Novedades); `BOTH` (Inicio y Novedades)	Sección del sitio donde aparece.
`SocialStatus`	`DETECTED` (Detectada, pendiente de moderar); `LINKED` (Aprobada y vinculada a una publicación); `DISCARDED` (Descartada)	Estado de moderación de una publicación de red.
`NoticeStatus`	`ACTIVE` (Activo); `PAUSED` (Pausada)	Estado de un aviso.
`NoticeAudience`	`ALL` (Todos); `STUDENTS` (Estudiantes); `ASPIRANTS` (Aspirantes); `GRADUATES` (Egresados)	Destinatarios de un aviso.
`DocumentKind`	`SCHEDULE` (Horario); `EXAM_CALENDAR` (Calendario de exámenes); `RESOLUTION` (Resolución); `PLAN` (Plan de estudios); `NOTICE_ATTACHMENT` (Adjunto de aviso); `OTHER` (Otro)	Tipo de documento.
`DocumentStatus`	`ACTIVE` (Activo); `ARCHIVED` (Archivado)	Estado de un documento.
`TermKind`	`FIRST` (1.er cuatrimestre); `SECOND` (2.º cuatrimestre); `EXTRAORDINARY` (Extraordinario)	Tipo de ciclo.
`MessageStatus`	`NEW` (Nuevo); `READ` (Leído); `ANSWERED` (Respondido)	Estado de un mensaje de contacto.

Relaciones entre tablas

Relación	Cardinalidad	Regla al borrar
`Role` → `RolePermission` (por `roleId`)	1 a N	En cascada: se borran los hijos
`Role` → `User` (por `roleId`)	1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`User` → `Session` (por `userId`)	1 a N	En cascada: se borran los hijos
`Category` → `Publication` (por `categoryId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`SocialPost` → `Publication` (por `socialPostId`)	1 a 0..1	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`Category` → `Notice` (por `categoryId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`Document` → `Notice` (por `attachmentId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`AcademicTerm` → `Schedule` (por `termId`)	1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)

Relación	Cardinalidad	Regla al borrar
`Document` → `Schedule` (por `documentId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`AcademicTerm` → `ExamCall` (por `termId`)	1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`Document` → `ExamCall` (por `documentId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`Gallery` → `ContentSection` (por `galleryId`)	0..1 a N	Se deja vacío el campo (SetNull)
`Gallery` → `GalleryItem` (por `galleryId`)	1 a N	En cascada: se borran los hijos
`User` → `AuditLog` (por `userId`)	0..1 a N	Se deja vacío el campo (SetNull)

6. Configuración inicial

La configuración vive en variables de entorno (archivo `.env` en la raíz; se parte de `.env.example`) y se **valida al arrancar**: si falta algo que el proveedor elegido exige, la API no arranca y dice cuáles faltan.

Variable	Valor de desarrollo	Para qué
`DATABASE_URL`	`postgresql://ifts2:ifts2@localhost:5433/ifts2`	Conexión a la base
`TEST_DATABASE_URL`	`…/ifts2_test`	Base de las pruebas (se migra sola)
`APP_ORIGIN` / `API_ORIGIN` / `API_PORT`	`http://localhost:3000` / `:4000` / `4000`	Orígenes (CORS, cookies, OAuth) y puerto de la API
`TRUST_PROXY`	`0`	Cantidad de proxies de confianza (hosting)
`SESSION_COOKIE_NAME`, `SESSION_TTL_HOURS`, `SESSION_REMEMBER_DAYS`	`ifts2_session`, 8, 14	Sesión (8 horas; 14 días con «recordarme»)
`LOGIN_MAX_ATTEMPTS`, `LOGIN_WINDOW_MINUTES`	5, 15	Bloqueo tras intentos fallidos (por IP y por correo)
`APP_ENCRYPTION_KEY`	clave de ejemplo	**32 bytes en base64**. Cifra los secretos en la base. **Cambiala en producción**
`STORAGE_PROVIDER`	`local`	`local` (carpeta `uploads/`), `memory` (pruebas) o `s3` (compatible S3)
`STORAGE_LOCAL_DIR`, `STORAGE_SIGNING_SECRET`, `STORAGE_SIGNED_URL_TTL_SECONDS`	`uploads`, secreto, 300	Archivos locales y su URL firmada (el secreto es obligatorio en producción)
`S3_*`	—	Solo con `s3`: `S3_BUCKET`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY` obligatorias; `S3_ENDPOINT`, `S3_REGION`, `S3_PUBLIC_URL`, `S3_FORCE_PATH_STYLE`
`MAILER_PROVIDER`, `SMTP_URL`, `CONTACT_TO_EMAIL`	`console`, —, —	Valores por defecto del correo (el cliente lo cambia desde el panel)

Variable	Valor de desarrollo	Para qué
`INSTAGRAM_PROVIDER`, `INSTAGRAM_APP_ID`, `INSTAGRAM_APP_SECRET`, `INSTAGRAM_REDIRECT_URI`	`fake`, —, —, —	Valores por defecto de Instagram (el cliente los carga desde el panel)
`CRON_SECRET`	vacío (deshabilitado)	Secreto (≥ 16 caracteres) para que un cron externo llame a `POST /social/cron`
`SOCIAL_SCHEDULER_ENABLED`, `SOCIAL_SYNC_INTERVAL_HOURS`	`false`, 24	Planificador interno de sincronización (alternativa al cron externo)
`REVALIDATE_SECRET`	`dev-only-revalidate-secret`	Secreto compartido API ↔ web para refrescar la caché al instante (mismo valor en ambos)
`LOG_LEVEL`	`info`	Nivel de logs
`SEED_MODE`, `SEED_ADMIN_EMAIL`, `SEED_ADMIN_PASSWORD`	`demo`, `admin@ifts2.local`, `Cambiame-123!`	Semilla: `demo` o `prod`; datos del administrador inicial

6.1 Colores y tipografías

Los valores salen de `docs/specs/design-tokens.json`, `pnpm tokens` los vuelca a `apps/web/src/styles/tokens.css` y los componentes usan solo clases del tema (`bg-primary-strong`, `text-accent`, `font-display`). **Escribir un hex o un nombre de fuente en un componente rompe `pnpm lint`.** La tipografía se elige en un único lugar: `apps/web/src/app/fonts.ts`.

7. Integraciones, despliegue y operación

7.1 Instagram (la configura el cliente desde el panel)

Panel → **Configuración** → **Integraciones** (permiso `integrations:manage`; lo trae ADMIN por defecto). Los datos se guardan cifrados (AES-256-GCM) y **nunca** vuelven en una respuesta, un log ni la auditoría: la pantalla muestra máscaras (`••••1234`).

Guía para el cliente (la misma que muestra la pantalla; texto en `packages/shared/src/instagram-guide.ts`):

1. La cuenta de Instagram del instituto debe ser **profesional** (Empresa o Creador).
2. Crear una app en **Meta for Developers** (developers.facebook.com) con el caso de uso de Instagram.
3. Agregar el producto **Instagram** (inicio de sesión de Instagram).
4. Registrar la **dirección de redirección** que muestra la pantalla (`<API_ORIGIN>/api/v1/integrations/instagram/callback`) en «URI de redireccionamiento de OAuth válidos».
5. Copiar **App ID** y **App Secret** y pegarlos en la pantalla.
6. Pulsar **Conectar con Instagram**, iniciar sesión con la cuenta del instituto, aceptar y usar **Probar conexión**. Alternativa: pegar un token de larga duración.

Estados: sin configurar (modo demostración) · credenciales guardadas, falta conectar · conectada · token por vencer · error. El token de larga duración dura ~60 días y se **renueva automáticamente** cuando faltan 10 o menos (`REFRESH_WHEN_DAYS_LEFT`). La sincronización busca las publicaciones con el **hashtag** configurado dentro de un rango de fechas, es **idempotente** por `externalId` y deja todo como «pendiente»: nada se publica sin moderación humana.

Para que la sincronización sea diaria hay dos opciones: (A, recomendada en hosting gratuito) un **cron externo** que llame una vez por día a `POST /api/v1/social/cron` con `X-Cron-Secret: <CRON_SECRET>`, o (B) el planificador interno (`SOCIAL_SCHEDULER_ENABLED=true`, solo si el servidor está siempre encendido).

7.2 Correo (formulario de contacto)

Panel → Configuración → Integraciones → Correo: correo de destino y, opcionalmente, un SMTP (`smtp://usuario:clave@host:587`). Con **Probar** se envía un mensaje de prueba. Sin SMTP el correo «sale» por consola (modo demostración). **El mensaje de contacto se guarda siempre en la base antes de intentar enviar el correo**, así no se pierde si el correo falla.

7.3 Almacenamiento de archivos

`STORAGE_PROVIDER=local` guarda en `uploads/` (no se versiona ni entra en el `.zip`); `s3` usa cualquier servicio compatible (Supabase Storage, Cloudflare R2, MinIO, AWS). Los archivos se descargan siempre con **URL firmada** de corta duración; la clave de almacenamiento no sale de la API. Imágenes: se convierten a WebP de hasta 1600 px; el tipo se verifica por la **firma real** del archivo (no por la extensión).

7.4 Despliegue (procedimiento general)

El hosting definitivo **está por definir con el cliente** (se busca uno gratuito, sin vencimiento). El procedimiento es el mismo en cualquier proveedor:

1. Provisionar PostgreSQL 17 y definir `DATABASE_URL`.
2. Definir las variables de producción: `NODE_ENV=production`, `APP_ORIGIN` / `API_ORIGIN` reales (HTTPS), `APP_ENCRYPTION_KEY` propia, `STORAGE_SIGNING_SECRET`, `REVALIDATE_SECRET` (igual en API y web), `TRUST_PROXY` según el proveedor, `SEED_MODE=prod`, `SEED_ADMIN_EMAIL` y `SEED_ADMIN_PASSWORD`.
3. `pnpm install --frozen-lockfile`, `pnpm build`, `pnpm exec prisma migrate deploy` y `SEED_MODE=prod pnpm db:seed`.
4. Levantar la API (`pnpm --filter @ifts2/api start`) y la web (`pnpm --filter @ifts2/web start`, con `PORT`). La cookie de sesión es `Secure` en HTTPS.
5. Cargar la configuración de Instagram y correo desde el panel y programar el cron diario.
6. Hacer copias de seguridad de la base y de `uploads/` (sección 2.6).

El repositorio incluye `.github/workflows/ci.yml` (lint, tipos, pruebas con PostgreSQL de servicio, build y Cypress).

8. Componentes principales

8.1 Núcleo de la API

Componente	Ubicación	Responsabilidad y métodos clave
`createApp(opts)` / `getContext(app)`	`src/app.ts`	Arma Express: helmet, CORS, parseo con límite de tamaño, `originCheck`, `attachAuth`, montaje de módulos, 404 y manejador de errores
`loadConfig(env)`	`src/config.ts`	Valida el entorno con zod; si falta algo lanza `ConfigError` y la API no arranca
`AppContext` (`req.ctx`)	`src/context.ts`	`prisma`, `storage`, `getMailer()`, `getInstagramClient()`, `fetch` inyectable, limitadores, logger

Componente	Ubicación	Responsabilidad y métodos clave
`attachAuth`, `requireAuth`, `requirePermission(action)`, `requireAnyPermission(...)`	`src/middleware/auth.ts`	Resuelven la sesión desde la cookie (hash del token en la tabla `Session`) y exigen permiso: 401 sin sesión, 403 sin permiso
`validate({ body, query, params })` / `valid<T>()`	`src/middleware/validate.ts`	Valida con zod y deja el dato tipado; los errores salen por campo
`originCheck`	`src/middleware/origin.ts`	Defensa CSRF: en métodos que modifican datos exige que `Origin` sea `APP_ORIGIN`
`createLimiters(config)`	`src/middleware/rate-limit.ts`	Login (5 fallos / 15 min por IP y por correo), contacto (5 por hora), sincronización (6 por minuto)
`revalidateWeb`	`src/middleware/revalidate-web.ts`	Avisa a la web tras cada modificación para refrescar la caché
`errorHandler`	`src/middleware/error-handler.ts`	Convierte cualquier error en Problem Details (RFC 7807)
`audit.fromReq(req, entry, tx?)`	`src/modules/audit/audit.service.ts`	Registra quién hizo qué, dentro de la misma transacción que la modificación
`hashPassword`, `verifyPassword`, `generateTempPassword`	`src/lib/password.ts`	argon2id; claves temporales aleatorias de 14 caracteres
`encrypt`, `decrypt`, `encryptJson`, `maskSecret`	`src/infra/crypto.ts`	AES-256-GCM con IV aleatorio (formato versionado); detecta alteraciones; enmascara secretos
`StorageProvider` (`LocalDisk`, `InMemory`, `S3`)	`src/infra/storage/`	Guardar, leer, borrar y firmar URL de descarga; `storeUpload` valida firma real, límites y convierte imágenes a WebP
`Mailer` (`ConsoleMailer`, `SmtpMailer`, `FakeMailer`)	`src/infra/mailer/`	Enviar mensajes; el proveedor se resuelve desde el panel o el entorno
`InstagramClient` (`FakeInstagramClient`, `GraphInstagramClient`)	`src/infra/instagram/`	`fetchMedia` (paginado y filtrado por hashtag), `testConnection`, canje de código OAuth y renovación del token; solo sigue URLs de `graph.instagram.com`
`IntegrationStore`	`src/infra/integration-store.ts`	Lee y guarda la configuración cifrada de Instagram y correo

8.2 Reglas de negocio destacadas

Regla	Dónde	Qué hace
Vigencia	`packages/shared/src/visibility.ts`	`isNoticeVisible`, `isPublicationVisible`, `isNoticeExpiringSoon`, `isScheduleVisible`, `isExamCallVisible`: reciben `now` (testeables). `visibleFrom` inclusivo, `visibleUntil` exclusivo; un fin dado como día vale hasta 23:59:59.999 hora de Buenos Aires
Un único ciclo público	`modules/terms/terms.service.ts`	`setPublicTerm` apaga el anterior y enciende el nuevo en una transacción; además hay un índice único parcial en la base (`AcademicTerm_publicCurrent_unique`)
Estado «vencido» calculado	`modules/notices/notices.service.ts`	`viewStatusOf` lo deduce de las fechas; **no se guarda**. `canDeleteNotice`: solo se elimina lo no vigente

Regla	Dónde	Qué hace
Eliminar archivos	`modules/documents/documents.service.ts`	`findReferences` lista dónde se usa un documento; si está en uso, eliminar responde 409 con la lista
Moderación de redes	`modules/social/social.service.ts`	`approvePost` (exige consentimiento, copia la imagen al almacenamiento propio y crea la publicación), `discardPost`, `restorePost`, `createManualPost`, `deletePost`
Sincronización	`modules/social/sync.service.ts`, `jobs.ts`, `token.service.ts`	`syncInstagram` (idempotente por `externalId`), `runSocialJob` (cron o planificador), `renewInstagramTokenIfNeeded`
Sitio público con vigencia	`modules/public/public.service.ts`	Aplica la vigencia en dos capas: filtro SQL y verificación con las funciones compartidas; devuelve `downloadUrl: null` si falla el almacenamiento

8.3 Web

Componente	Ubicación	Responsabilidad
`middleware.ts`	`src/middleware.ts`	Sin cookie de sesión, `/panel/*` redirige al login (la validación real la hace `requireSession()` y la API)
`PublicLayout`, `PanelShell`	`src/components/layout/`	Estructura del sitio y del panel (menú lateral, drawer móvil, contadores, opciones según el rol)
Kit de UI (`Button`, `Dialog`, `Table`, `Toast`, …)	`src/components/ui/`	Componentes accesibles sobre Radix; catálogo en `/ui-kit`
`ConfirmDialog` / `useConfirm()`	`src/components/ui/confirm-dialog.tsx`	**Único** diálogo de confirmación del producto (eliminar, pausar, publicar, cambiar ciclo…)
Cliente `api`	`src/lib/api.ts`	`get/post/put/patch/delete` con cookies; convierte Problem Details en `ApiError` con `fieldErrors` y mensaje para la persona
`features/public/`, `features/panel-a/`, `features/panel-b/`	`src/features/`	Pantallas; cada una tiene un README con el mapa pantalla → endpoints → componentes
Revalidación	`src/app/api/revalidate/route.ts`	Invalida la caché ISR (`public`, `settings`) cuando la API lo pide

8.4 Cómo agregar un módulo de API

Crear `apps/api/src/modules/<nombre>/index.ts` con `mountPath` y `router`; usar `requirePermission`, `validate` con un esquema de `@ifts2/shared`, `req.ctx` para los servicios y `audit.fromReq` en cada modificación. La plantilla completa, con paginación y pruebas, está en `apps/api/README.md`.

9. Gestión de errores y validaciones

9.1 Errores: Problem Details (RFC 7807)

Todo error de la API tiene el mismo formato (`application/problem+json`):

```
{
  "type": "about:blank#validation",
  "title": "Hay datos incorrectos",
  "status": 400,
  "code": "validation",
  "detail": "El título es obligatorio.",
  "errors": { "title": ["El título es obligatorio."] }
}
```

Estado	`code`	Cuándo ocurre
400	`validation`	Datos inválidos (errores por campo en `errors`)
401	`unauthorized`	Sin sesión o credenciales incorrectas (mensaje genérico, no revela cuál)
403	`forbidden`	Sin permiso para la acción, o cambio de clave pendiente
404	`not-found`	No existe
409	`conflict`	Estado incompatible: eliminar algo vigente o en uso, correo duplicado
413 / 415	`payload-too-large` / `unsupported-media-type`	Archivo demasiado grande o de un tipo no permitido
422	`unprocessable`	Regla de negocio (p. ej. publicar un horario sin archivo)
429	`too-many-requests`	Límite de frecuencia (login, contacto, sincronización)
502	`bad-gateway`	Falla una integración externa (Instagram, correo) con mensaje claro
500	`internal`	Error inesperado: se registra en el log y la persona ve un mensaje genérico

9.2 Validaciones

- **Entrada**: siempre con **zod** (`packages/shared/src/schemas.ts`), con los mismos esquemas en el navegador (mensajes en línea) y en la API (la última palabra). Mensajes en español con concordancia de género y número («La fecha de fin debe ser posterior a la fecha de inicio.»).
- **Campos obligatorios y formatos**: título, texto, correo, URL, fechas, etc.; los textos se recortan (`trim`) antes de validar.
- **No duplicados**: correo de usuario único; ciclo (año, tipo) único; categoría (nombre, tipo) único; `externalId` de Instagram único; clave de almacenamiento única.
- **Límites**: PDF hasta 10 MB; imágenes hasta 5 MB (se reducen a 1600 px y WebP); tipos permitidos PDF, JPG, PNG y WebP, verificados por la firma del archivo.
- **Eliminar**: solo contenido no vigente (borrador, pausado, descartado, vencido o archivado) y sin referencias; si no, 409 con el motivo. En la interfaz, **siempre con diálogo de confirmación**.
- **Sesiones y claves**: argon2id; sesión revocable (en la base solo se guarda el hash del token); cookie `HttpOnly` + `SameSite=Lax` (+ `Secure` en HTTPS); 5 intentos fallidos cada 15 minutos por IP y por correo; claves temporales que se muestran una sola vez y obligan a cambiarlas.
- **Contacto**: campo señuelo (honeypot) contra bots, 5 envíos por hora por IP y consentimiento obligatorio (Ley 25.326).
- **Seguridad de cabeceras**: helmet en la API y cabeceras de seguridad en la web (`X-Frame-Options`, `Strict-Transport-Security`, CSP acotada, etc.).

- **SSRF**: la descarga de imágenes por URL rechaza direcciones internas.

9.3 Degradación: qué pasa cuando algo falla

Falla	Comportamiento
Instagram	La sincronización responde 502 con un mensaje claro y deja `lastSyncOk = false`; el panel muestra el estado de error; el sitio sigue mostrando lo ya aprobado; la **carga manual** sirve de contingencia
Correo	El mensaje de contacto **ya está guardado** y aparece en la bandeja del panel (`mailSent = false`); la persona que escribió ve la confirmación igual
Almacenamiento	Los archivos adjuntos se muestran sin enlace de descarga (`downloadUrl: null`) y el resto del sitio funciona
API caída	La web pública muestra estados de error amigables en cada sección en lugar de romperse (probado con Cypress contra una web compilada sin API)
Base de datos caída	`GET /health` informa `db` con error; el resto responde 500 genérico, sin detalles internos
Falta configuración	La API no arranca y lista las variables que faltan (falla temprano, no a mitad de camino)

10. Pruebas implementadas (resumen)

El detalle, con salidas reales y capturas, está en el documento **Evidencias de pruebas**. Resultado de la última verificación (30/10/2026):

Suite	Herramienta	Cantidad	Qué cubre
Unitarias de `packages/shared`	Vitest	43 pruebas (2 archivos)	Vigencia (bordes y zona horaria), permisos por rol, esquemas zod y mensajes
Unitarias de la API	Vitest	7 archivos en `test/unit`	Cifrado, configuración, almacenamiento, Markdown, cliente de Instagram, lógica pura
Integración de la API	Vitest + Supertest + PostgreSQL	18 archivos en `test/api`	Cada módulo contra la base real `ifts2_test`: permisos, validación, reglas y auditoría
Extremo a extremo (interfaz real)	Cypress 16 + axe	ver *Evidencias de pruebas*	Recorridos del sitio y del panel a 390 y 1440 px con accesibilidad (WCAG 2.2 AA)
Flujos de negocio y seguridad	`pnpm verify`	33 comprobaciones	Flujos H1–H8 y H11 de punta a punta, cabeceras, cookies, secretos fuera de logs
Carga	autocannon	400 usuarios virtuales	Uso real: p95 525 ms y 0 errores 5xx
Rendimiento y accesibilidad	Lighthouse móvil	4 páginas	Rendimiento 97–99, accesibilidad 98–100

Los totales exactos de la corrida de entrega (por archivo y por suite) están en el documento de evidencias y en `docs/evidencias/`.

11. Limitaciones conocidas

Lo que **no se pudo probar** o quedó pendiente, dicho sin rodeos:

- **Instagram real (Meta)**: no hubo credenciales de una cuenta profesional y una app de Meta. `GraphInstagramClient` se probó **solo contra un servidor HTTP simulado** (paginación, errores 190/429/500, fallas de red, canje y renovación de token) siguiendo la documentación oficial; falta la prueba de concepto en vivo. Mientras tanto la carga manual cubre la contingencia.
- **Correo SMTP real**: se probó con el correo falso y por consola; no se envió un correo real (falta definir el remitente con el cliente).
- **Almacenamiento S3 real**: `S3Storage` está implementado pero no probado contra un servicio real.
- **Carga**: se usó **autocannon**, no k6 (que figuraba en la planificación). En saturación (400 usuarios sin pausa) un solo proceso de Node alcanza su techo en ~300–360 pedidos por segundo y aparecen reinicios de conexión; para el uso real esperado (~133 pedidos/s) cumple. El techo se sube con CDN o más procesos.
- **Datos pendientes del cliente**: datos de ingreso (fechas y documentación) y ubicación de las materias por año del plan de estudios; los marcadores quedan como `[a confirmar]` en el sitio.
- **Nombres Ingreso / Inscripciones, hashtag y colores** quedaron a validar con el cliente. El hashtag `#ifts2web` es provisorio; la tipografía es Plus Jakarta Sans (confirmado; la Etapa 2 proponía Inter).
- **Documentación de la entrega del 30/10:** el PDF y las tablas de campos se generaron antes de los roles configurables (RF11). Hay que ejecutar `pnpm docs:der` y `pnpm docs:pdf` para regenerarlos con el esquema actual.
- `pnpm db:reset` y `pnpm e2e --reset` borran la base de desarrollo y deben ejecutarse por una persona.

12. Para continuar el proyecto

1. Leer `README.md` (visión de conjunto), `docs/BUILD-NOTES.md` (reglas de trabajo) y `docs/specs/AGENTS.md` (reglas no negociables).
2. Leer `docs/specs/00` a `10` en orden (visión, requerimientos con criterios de aceptación, decisiones, arquitectura, API, pantallas, pruebas, backlog, datos, preguntas abiertas, entregables).
3. Mirar `docs/avance.md` (qué se hizo, dificultades y pendientes) y `docs/decisiones/` (por qué se decidió cada cosa).
4. Trabajar con ramas por historia y commits convencionales (`CONTRIBUTING.md`). Definición de terminado: criterios de aceptación cumplidos, pruebas en verde, `pnpm lint` , `pnpm typecheck` y `pnpm build` sin errores, axe sin violaciones graves, verificado a 390 y 1440 px, toda modificación audita y la documentación al día.